

FULL-STACK LOGIN SYSTEM

Vercel + Render + MongoDB Atlas

Simple authentication project with a separate frontend and backend

1. What You Will Build

A simple login system where the frontend is deployed on Vercel, the backend API is deployed on Render, and user accounts are stored in MongoDB Atlas.

- Frontend: HTML, CSS, and JavaScript (deploy to Vercel).
- Backend: Node.js + Express (deploy to Render).
- Database: MongoDB Atlas.
- Authentication: email/password with bcrypt password hashing and JWT tokens.

2. Final Architecture

Browser → Vercel Frontend → Render Backend API → MongoDB Atlas

The browser never connects directly to MongoDB. The frontend sends HTTP requests to the backend. The backend validates credentials and communicates with MongoDB.

3. Project Structure

```
login-system/  
├── backend/  
│   ├── server.js  
│   ├── package.json  
│   ├── .env  
│   └── .gitignore  
└── frontend/  
    ├── index.html  
    ├── dashboard.html  
    ├── style.css  
    └── app.js
```

4. Step 1 — Create MongoDB Atlas Database

1. Create or sign in to a MongoDB Atlas account.
2. Create a free database/cluster.
3. Create a database user. Save the username and password securely.
4. Configure Network Access. For a simple deployment, allow the Render server to connect. During learning/testing, you may temporarily use 0.0.0.0/0, but restrict access in a production system where possible.
5. Get the MongoDB connection string. It normally looks like:
mongodb+srv://USERNAME:PASSWORD@CLUSTER.mongodb.net/login_db?retryWrites=true&w=majority
6. Do not place this connection string in frontend JavaScript or commit it to GitHub.

5. Step 2 — Create the Backend

Open a terminal:

```
mkdir login-system
cd login-system
mkdir backend
cd backend
npm init -y
npm install express mongoose bcryptjs jsonwebtoken cors dotenv
```

Create backend/server.js:

```
const express = require("express");
const mongoose = require("mongoose");
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");
const cors = require("cors");
require("dotenv").config();

const app = express();

app.use(cors({
  origin: process.env.FRONTEND_URL || "*"
}));
app.use(express.json());

const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true, lowercase: true },
  password: { type: String, required: true }
});

const User = mongoose.model("User", userSchema);

app.get("/", (req, res) => {
  res.json({ message: "Login API is running." });
});

app.post("/api/register", async (req, res) => {
  try {
    const { name, email, password } = req.body;

    if (!name || !email || !password) {
      return res.status(400).json({ message: "All fields are required." });
    }

    if (password.length < 6) {
      return res.status(400).json({ message: "Password must be at least 6 characters." });
    }

    const existingUser = await User.findOne({ email });
    if (existingUser) {
```

```

    return res.status(409).json({ message: "Email is already registered." });
  }

  const hashedPassword = await bcrypt.hash(password, 10);

  await User.create({
    name,
    email,
    password: hashedPassword
  });

  res.status(201).json({ message: "Registration successful." });
} catch (error) {
  res.status(500).json({ message: "Server error." });
}
});

app.post("/api/login", async (req, res) => {
  try {
    const { email, password } = req.body;

    const user = await User.findOne({ email });
    if (!user) {
      return res.status(401).json({ message: "Invalid email or password." });
    }

    const validPassword = await bcrypt.compare(password, user.password);
    if (!validPassword) {
      return res.status(401).json({ message: "Invalid email or password." });
    }

    const token = jwt.sign(
      { userId: user._id, name: user.name, email: user.email },
      process.env.JWT_SECRET,
      { expiresIn: "1h" }
    );

    res.json({
      message: "Login successful.",
      token,
      user: {
        name: user.name,
        email: user.email
      }
    });
  } catch (error) {
    res.status(500).json({ message: "Server error." });
  }
});

app.get("/api/profile", async (req, res) => {
  try {

```

```

const authHeader = req.headers.authorization;

if (!authHeader || !authHeader.startsWith("Bearer ")) {
  return res.status(401).json({ message: "Unauthorized." });
}

const token = authHeader.split(" ")[1];
const decoded = jwt.verify(token, process.env.JWT_SECRET);

res.json({
  message: "Protected data.",
  user: {
    name: decoded.name,
    email: decoded.email
  }
});
} catch (error) {
  res.status(401).json({ message: "Invalid or expired token." });
}
});

const PORT = process.env.PORT || 5000;

mongoose.connect(process.env.MONGODB_URI)
  .then(() => {
    console.log("MongoDB connected.");
    app.listen(PORT, () => {
      console.log(`Server running on port ${PORT}`);
    });
  })
  .catch((error) => {
    console.error("MongoDB connection failed:", error);
  });

```

6. Step 3 — Backend Environment Variables

Create backend/.env:

```

MONGODB_URI=your_mongodb_connection_string
JWT_SECRET=replace_with_a_long_random_secret
FRONTEND_URL=http://localhost:5500
PORT=5000

```

Create backend/.gitignore:

```

node_modules/
.env

```

7. Step 4 — Create the Frontend

Create frontend/index.html:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Simple Login</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <main class="card">
    <h1>Welcome Back</h1>
    <p class="subtitle">Sign in to your account</p>

    <form id="loginForm">
      <label>Email</label>
      <input type="email" id="email" required>

      <label>Password</label>
      <input type="password" id="password" required>

      <button type="submit">Login</button>
      <p id="message"></p>
    </form>

    <p>Don't have an account? <a href="#" id="showRegister">Register</a></p>

    <form id="registerForm" class="hidden">
      <h2>Create Account</h2>
      <label>Name</label>
      <input type="text" id="name">

      <label>Email</label>
      <input type="email" id="regEmail">

      <label>Password</label>
      <input type="password" id="regPassword">

      <button type="submit">Register</button>
      <p id="registerMessage"></p>
    </form>
  </main>

  <script src="app.js"></script>
</body>
</html>

```

Create frontend/style.css:

```

* {
  box-sizing: border-box;
  font-family: Arial, sans-serif;
}

```

```
body {  
  margin: 0;  
  min-height: 100vh;  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  background: #f2f4f7;  
}
```

```
.card {  
  width: 380px;  
  max-width: 90%;  
  padding: 32px;  
  background: white;  
  border-radius: 12px;  
  box-shadow: 0 8px 30px rgba(0,0,0,.10);  
}
```

```
h1, h2 {  
  margin-top: 0;  
}
```

```
.subtitle {  
  color: #666;  
}
```

```
label {  
  display: block;  
  margin: 14px 0 6px;  
}
```

```
input {  
  width: 100%;  
  padding: 12px;  
  border: 1px solid #ccc;  
  border-radius: 6px;  
}
```

```
button {  
  width: 100%;  
  margin-top: 18px;  
  padding: 12px;  
  border: 0;  
  border-radius: 6px;  
  background: #222;  
  color: white;  
  cursor: pointer;  
}
```

```
button:hover {  
  opacity: .9;  
}
```

```

}

.hidden {
  display: none;
}

#message, #registerMessage {
  min-height: 20px;
}

```

Create frontend/app.js:

```

const express = require("express");

const mongoose = require("mongoose");

const bcrypt = require("bcryptjs");

const jwt = require("jsonwebtoken");

const cors = require("cors");

require("dotenv").config();

const app = express();

app.use(cors());

// app.use(
//   cors({
//     origin: process.env.FRONTEND_URL || "",
//   }),
// );

app.use(express.json());

const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true, lowercase: true },
  password: { type: String, required: true },
});

```

```

const User = mongoose.model("User", userSchema);

app.get("/", (req, res) => {
  res.json({ message: "Login API is running." });
});

app.post("/api/register", async (req, res) => {
  try {
    const { name, email, password } = req.body;

    if (!name || !email || !password) {
      return res.status(400).json({ message: "All fields are required." });
    }

    if (password.length < 6) {
      return res
        .status(400)
        .json({ message: "Password must be at least 6 characters." });
    }

    const existingUser = await User.findOne({ email });

    if (existingUser) {
      return res.status(409).json({ message: "Email is already registered." });
    }

    const hashedPassword = await bcrypt.hash(password, 10);

    await User.create({
      name,
      email,

```

```

    password: hashedPassword,
  });

  res.status(201).json({ message: "Registration successful." });
} catch (error) {
  res.status(500).json({ message: "Server error." });
}
});

app.post("/api/login", async (req, res) => {
  try {
    const { email, password } = req.body;

    const user = await User.findOne({ email });

    if (!user) {
      return res.status(401).json({ message: "Invalid email or password." });
    }

    const validPassword = await bcrypt.compare(password, user.password);

    if (!validPassword) {
      return res.status(401).json({ message: "Invalid email or password." });
    }

    const token = jwt.sign(
      { userId: user._id, name: user.name, email: user.email },
      process.env.JWT_SECRET,
      { expiresIn: "1h" },
    );

    res.json({

```

```

    message: "Login successful.",
    token,
    user: {
      name: user.name,
      email: user.email,
    },
  });
} catch (error) {
  res.status(500).json({ message: "Server error." });
}
});

app.get("/api/profile", async (req, res) => {
  try {
    const authHeader = req.headers.authorization;

    if (!authHeader || !authHeader.startsWith("Bearer ")) {
      return res.status(401).json({ message: "Unauthorized." });
    }

    const token = authHeader.split(" ")[1];
    const decoded = jwt.verify(token, process.env.JWT_SECRET);

    res.json({
      message: "Protected data.",
      user: {
        name: decoded.name,
        email: decoded.email,
      },
    });
  }
});

```

```

    } catch (error) {

      res.status(401).json({ message: "Invalid or expired token." });

    }

  });

```

```
const PORT = process.env.PORT || 5000;
```

mongoose

```

.connect(process.env.MONGODB_URI)

.then(() => {

  console.log("MongoDB connected.");

  app.listen(PORT, () => {

    console.log(`Server running on port ${PORT}`);

  });

})

.catch((error) => {

  console.error("MongoDB connection failed:", error);

});

```

Create frontend/dashboard.html:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dashboard</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <main class="card">
    <h1>Dashboard</h1>
    <p id="welcome">Loading...</p>
    <button id="logout">Logout</button>
  </main>

  <script>
    const API_URL = "http://localhost:5000";

```

```

const token = localStorage.getItem("token");

if (!token) {
  window.location.href = "index.html";
}

fetch(`${API_URL}/api/profile`, {
  headers: {
    Authorization: `Bearer ${token}`
  }
})
.then(response => {
  if (!response.ok) throw new Error();
  return response.json();
})
.then(data => {
  document.getElementById("welcome").textContent =
    `Welcome, ${data.user.name} (${data.user.email})`;
})
.catch(() => {
  localStorage.removeItem("token");
  window.location.href = "index.html";
});

document.getElementById("logout").addEventListener("click", () => {
  localStorage.removeItem("token");
  window.location.href = "index.html";
});
</script>
</body>
</html>

```

8. Step 5 — Test the Backend Locally

From the backend folder:

```
node server.js
```

If successful, you should see messages indicating that MongoDB is connected and the server is running. Open <http://localhost:5000> in a browser. It should return the API status message.

9. Step 6 — Test the Frontend Locally

Use VS Code Live Server or another local static server to serve the frontend. Do not simply open the HTML file with `file://` if your browser blocks requests. The frontend `API_URL` should point to `http://localhost:5000` during local testing.

10. Step 7 — Put the Project on GitHub

Create a GitHub repository and push the project. Before pushing, verify that `.env` is ignored. Never upload `MONGODB_URI` or `JWT_SECRET`.

Typical commands:

```
git init
git add .
git commit -m "Initial login system"
git branch -M main
git remote add origin YOUR_GITHUB_REPOSITORY
git push -u origin main
```

11. Step 8 — Deploy the Backend to Render

1. Create a Render account and connect your GitHub repository.
2. Create a new Web Service.
3. Select the repository containing the backend.
4. Set the Root Directory to backend if the backend is inside the backend folder.
5. Build Command: npm install
6. Start Command: node server.js
7. Add environment variables in Render: MONGODB_URI, JWT_SECRET, FRONTEND_URL.
8. Deploy the service.
9. Copy the Render URL, for example <https://your-api.onrender.com>.

12. Step 9 — Update the Frontend for Render

Change API_URL in frontend/app.js and dashboard.html from localhost to your Render URL:

```
const API_URL = "https://your-api.onrender.com";
```

Commit and push the change to GitHub.

13. Step 10 — Deploy the Frontend to Vercel

1. Create/sign in to a Vercel account and import the GitHub repository.
2. If the frontend is inside a frontend folder, set the Vercel Root Directory to frontend.
3. For this plain HTML/CSS/JavaScript project, no special build command is required.
4. Deploy the project.
5. Copy the Vercel URL.

14. Step 11 — Fix CORS for the Vercel Domain

Go back to Render → Environment Variables. Change FRONTEND_URL from the localhost address to the exact Vercel URL, for example <https://your-login.vercel.app>. Redeploy/restart the Render service if required.

This allows the browser to call the Render API from your Vercel frontend.

15. Step 12 — Test the Complete System

- Open the Vercel URL.
- Register a new account.
- Open MongoDB Atlas and confirm that the user document exists.
- Confirm that the stored password is a bcrypt hash, not the original password.
- Log in using the registered account.
- Confirm that the dashboard loads protected user information.
- Click Logout.
- Try opening dashboard.html again. It should redirect to the login page.

16. Common Problems and Fixes

Problem	Fix
CORS error	Check FRONTEND_URL on Render. It must exactly match the Vercel origin, without an extra path.
MongoDB connection failed	Check MONGODB_URI, database user credentials, and MongoDB Atlas Network Access.
Cannot connect to server	Check that the Render service is running and that API_URL points to the Render URL.
404 on API	Confirm the route starts with /api and that the frontend uses the correct Render URL.
Environment variable undefined	Add the variable in Render's Environment Variables. Do not rely on your local .env file.
Login works locally but not on Vercel	Change API_URL from localhost to the Render HTTPS URL and check CORS.
Render service sleeps	Free hosting may have startup delays depending on the current Render plan.

17. Important Security Notes

- Never store plain-text passwords. This project hashes passwords with bcrypt.
- Never put MONGODB_URI or JWT_SECRET in frontend code.
- Never commit .env to GitHub.
- Use HTTPS in production. Vercel and Render provide HTTPS URLs.
- Use a strong, unpredictable JWT_SECRET.
- For a production application, consider HttpOnly secure cookies instead of storing JWTs in localStorage.
- Add rate limiting, stronger validation, password reset, email verification, and account lockout protections for a real production system.

18. Connection Summary

Vercel Frontend

↓ HTTPS / fetch()

Render Express API

↓ Mongoose

MongoDB Atlas

Registration: Frontend → POST /api/register → bcrypt hash → MongoDB

Login: Frontend → POST /api/login → bcrypt compare → JWT → Frontend

Protected page: Frontend → Authorization: Bearer TOKEN → Render → JWT verification → Response

19. Recommended Next Improvements

- Separate login and registration into different pages.
- Add a proper navigation/dashboard page.
- Add user roles such as Admin and User.
- Add CRUD functionality after authentication.
- Use environment variables for the API URL rather than hard-coding it.
- Move JWT authentication to secure HttpOnly cookies for a stronger production implementation.
- Add a logout endpoint if server-side token invalidation is required.